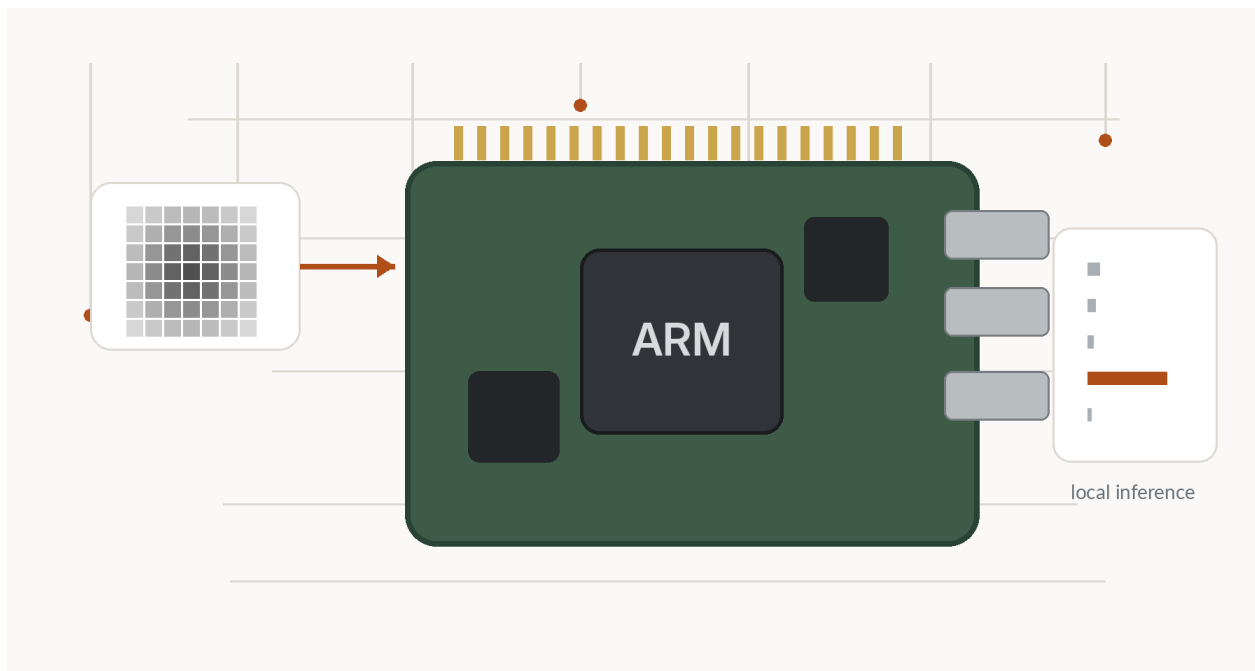


POSTGRADUATE PROGRAMME IN EMBEDDED SYSTEMS

Artificial Intelligence in Embedded Systems

PRACTICAL LABORATORY 01

From device to inference: first contact with Edge AI



12 September 2026 | Tiago Fonseca | ISEP

LAB PURPOSE

Run a previously trained AI model locally on a Raspberry Pi, understand what data and runtime assumptions make that result meaningful, and gather evidence rather than simply obtaining an output.

1

COURSE ORIENTATION

What this course is about, who is teaching it, and how the practical work will progress.

1.1 What this course is about

Artificial Intelligence in Embedded Systems sits at the boundary between machine learning and systems engineering. A model that performs well in a notebook is not yet an embedded AI solution. It must run on a target device, respect an input/output contract, fit within resource constraints, behave acceptably under real data, and provide evidence that supports an engineering decision.

Across the course, the central question is therefore not "which model has the highest accuracy?" but "which complete solution is appropriate for this deployment scenario, and what evidence supports that choice?"

COURSE PERSPECTIVE

Model quality and system behaviour are evaluated together. Accuracy, latency, memory, model size, robustness, reproducibility and deployment constraints will progressively become part of the same engineering argument.

1.2 Instructor



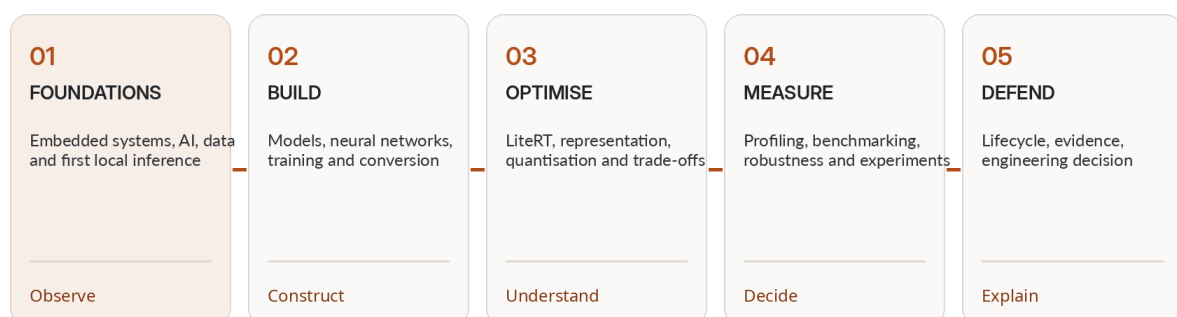
Tiago Fonseca

Lecturer and researcher

Lecturer at ISEP and PhD researcher at FEUP. His work focuses on multi-agent reinforcement learning, energy communities and deployable AI systems, with particular interest in how intelligent models move from research environments into constrained real systems.

Contact: tcf@isep.ipp.pt

1.3 How the course progresses



Progressive autonomy: the instructions become less prescriptive as the course moves from execution to engineering judgement.

The same logic carries across the practical sessions: first observe and execute, then build and convert, then optimise and measure, and finally design experiments and defend a deployment decision. PL1 deliberately starts with a model that is already trained so that the deployment stack can be examined before training complexity is introduced.

1.4 Working groups and Raspberry Pi kits

Form the number of working groups announced by the instructor. The number of groups is determined by the Raspberry Pi boards available for the session.

Group record	Complete in class
Number of groups announced	N = _____
Your group number	_____
Students	_____
Assigned board / kit ID	_____
Hostname, once identified	_____

1.5 Board handling rules

The Raspberry Pi is laboratory equipment. Treat the kit as a shared technical asset, not as a disposable development board.

- Use the board and power supply assigned to your group. Do not exchange components between labelled kits unless instructed.
- Keep liquids and food away from the board. Handle the board by the case or edges and avoid touching exposed contacts unnecessarily.
- Do not remove the microSD card, disconnect power abruptly, alter boot firmware, re-image the card or perform system-wide changes that are not required by the laboratory.
- Use a clean software shutdown before disconnecting power: ``sudo shutdown -h now``.
- Install Python packages inside your own virtual environment. Do not use ``sudo pip`` or modify the system Python installation.
- If a connector, cable, fan, storage card or board appears damaged or behaves unexpectedly, stop and report it rather than forcing the hardware.
- At the end of every practical class, return the Raspberry Pi, power supply and all accessories to the matching labelled box and hand the complete kit directly to the instructor.

RESPONSIBILITY

Each group is responsible for the integrity of its assigned kit during the session and for returning the complete kit at the end.

2

BACKGROUND FOR LABORATORY 1

A compact conceptual guide to embedded systems, AI at the edge and model deployment.

2.1 What is an embedded system?

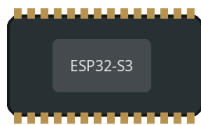
An embedded system is a computing system designed as part of a larger product, machine or process, usually with a specific operational purpose. It often interacts with the physical world through sensors and actuators and is designed under constraints that are less visible on a general-purpose workstation: memory, compute, power, thermal behaviour, storage, real-time deadlines, connectivity, reliability and cost.

Embedded does not mean "small" in a single sense. The term spans low-power microcontrollers, Linux single-board computers and specialised edge AI computers. What matters is the deployment context and the constraints imposed by the surrounding system.

A spectrum of embedded and edge targets

MCU

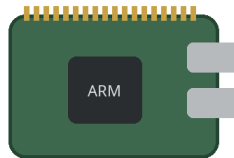
ESP32-S3



- 2 x Xtensa LX7, up to 240 MHz
- 512 KB on-chip SRAM
- Vector extensions for AI/DSP

SBC

Raspberry Pi 5



- 4 x Arm Cortex-A76, 2.4 GHz
- 1-16 GB LPDDR4X (variant)
- General-purpose Linux SBC

EDGE AI COMPUTER

Jetson Orin Nano Super



- 6 x Arm Cortex-A78AE
- 8 GB LPDDR5
- 67 INT8 TOPS; 7-25 W

Do not rank devices using a single headline number.

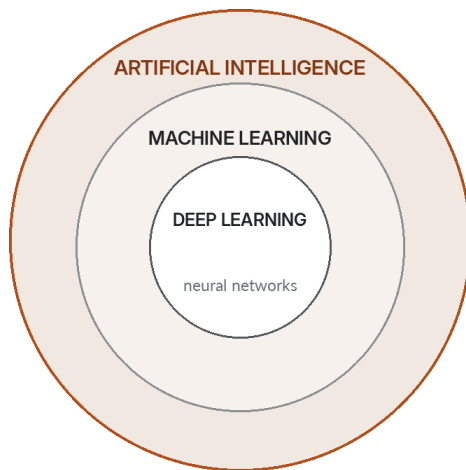
Clock frequency, RAM capacity, TOPS, memory bandwidth, accelerators, power envelope and software support describe different constraints. The engineering question is whether a workload satisfies its requirements on a particular target under a controlled protocol.

Reference platforms illustrate the spectrum; the assigned Raspberry Pi may be a different model. Specifications are not directly comparable by one metric.

DISCUSSION PROMPT

Suppose the same classifier can technically run on all three platforms. What additional requirements would you need before deciding which platform is appropriate for a real product?

2.2 AI, machine learning and deep learning: the minimum map



Two different computational phases

TRAINING

Data + optimisation -> learned parameters

Usually compute- and memory-intensive. Often performed on workstations, servers or accelerators.

INFERENCE

Input + trained model -> prediction / decision

This is the main focus of PL1: execute a previously trained model on the target device and inspect the full input/output contract.

Artificial intelligence is the broader field. Machine learning is one way of building AI systems by fitting models from data. Deep learning is a family of machine-learning methods based on multi-layer neural networks. Embedded AI can use any of these approaches; the laboratory sequence focuses primarily on learned models and their deployment.

KEY DISTINCTION

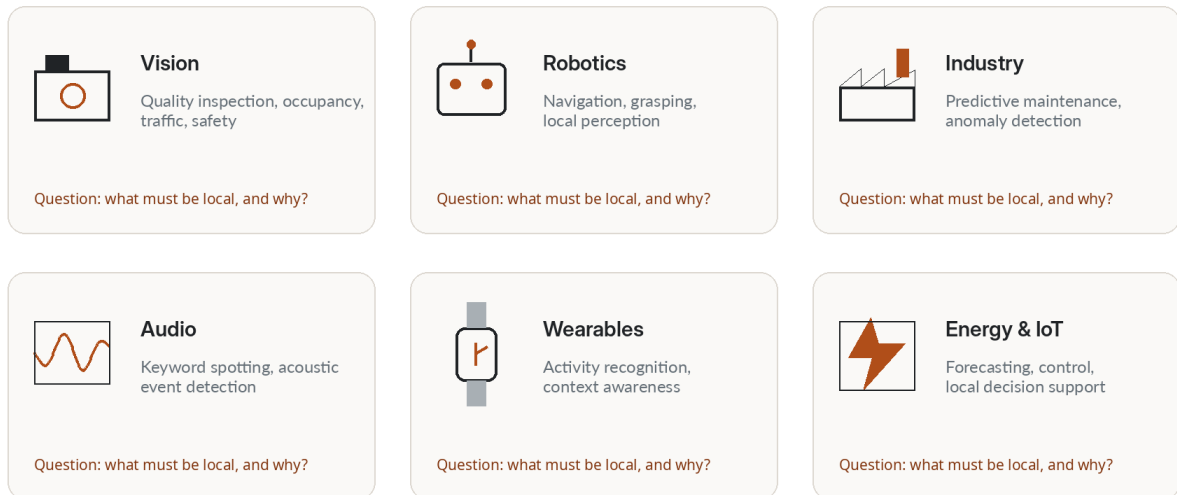
Training changes model parameters. Inference uses fixed learned parameters to produce outputs for new inputs. Training and inference therefore have different computational profiles and can happen on different machines.

2.3 Why run AI at the edge?

Running inference close to the data source can reduce communication latency, preserve data locally, keep a system useful when connectivity is unavailable and reduce bandwidth. It can also create harder constraints: less compute, less memory, limited power, heat dissipation and device-specific runtime support.

Potential reason for local inference	Engineering question that follows
Latency	What end-to-end response deadline must the complete pipeline meet?
Privacy / data locality	Which data must never leave the device, and what metadata may still be transmitted?
Intermittent connectivity	What behaviour is required when the network disappears?
Bandwidth / cloud cost	How much raw data would otherwise need to be transmitted?
Autonomy / control	Does the local decision affect a physical process that cannot wait for a remote service?

2.4 Where embedded AI appears



The deployment problem changes with the application. A visual inspection station may tolerate a larger model but require consistent throughput; a wearable may prioritise energy; a robot may care about tail latency and robustness; an industrial monitor may care about false negatives and long-term stability. The model cannot be judged independently of that context.

2.5 From a trained model to an embedded AI system

The deployment boundary

Training produces parameters. Deployment turns those parameters into an executable contract on a target system.



PL1 starts at the export/deploy boundary: the model is already trained, but you must recover and respect the assumptions that make inference meaningful

MODEL FILE ≠ COMPLETE SYSTEM

Preprocessing, runtime, hardware, labels, thresholds and provenance are part of the deployed behaviour.

The boundary between model development and embedded deployment is where many silent failures appear. The file can be valid, the runtime can execute, and the prediction can still be wrong because the deployed preprocessing, labels, tensor shapes, numeric types or interpretation do not match the training pipeline.

2.6 LiteRT and the .tflite artifact

Most machine-learning models are developed in a full training framework such as TensorFlow/Keras or PyTorch. Those frameworks are designed for experimentation and training: they manage model definitions, automatic differentiation, optimisers, datasets, checkpoints and often GPU acceleration. An embedded deployment normally does not need that complete training stack. It needs a compact representation of the trained computation and a runtime able to execute that computation efficiently on the target device.

LiteRT is Google's on-device inference runtime and deployment ecosystem, formerly known as TensorFlow Lite. It is therefore a deployment layer rather than a replacement for TensorFlow or PyTorch. Models trained with TensorFlow/Keras can be exported to LiteRT, and PyTorch models can also be converted through Google's LiteRT/PyTorch conversion tooling. The common result is a `.tflite` FlatBuffer artifact containing the computation graph, learned parameters and tensor interface required for inference. The Raspberry Pi executes that artifact with LiteRT without loading the full training framework.

Stage	Typical tools	What exists at this stage
Develop and train	TensorFlow / Keras, PyTorch	Training code, data pipeline, model definition, gradients, optimiser, checkpoints and evaluation logic.
Export / convert	Keras LiteRT export, TensorFlow conversion tools, LiteRT Torch / PyTorch conversion tools	A deployment artifact is produced and, when required, transformed or optimised for the target runtime.
Run on the device	LiteRT + <code>.tflite</code> artifact	Inference only: allocate tensors, copy input data, execute the graph and read outputs. The full training framework is not required on the Raspberry Pi.

Inference is a contract, not a function call

A model can execute without error and still be semantically wrong if any stage disagrees with training-time assumptions.



PL1 experiment: keep the model fixed and intentionally violate preprocessing. If predictions change, the failure belongs to the pipeline, not to the CPU.

The first laboratory will expose this contract explicitly rather than hiding it behind a high-level `predict()` call.

TERMINOLOGY

You will still encounter "TensorFlow Lite", "TFLite" and `tf.lite` in older tutorials, papers and code. LiteRT is the current name of the on-device runtime. The `.tflite` extension remains common because it names the deployed model artifact format. In new Python code for this course we use `ai_edge_liter.interpreter.Interpreter`.

QUESTION TO CARRY INTO THE PRACTICAL

If a `.tflite` file is copied to a Raspberry Pi and the runtime loads it successfully, what has actually been demonstrated - and what has not yet been demonstrated?

3

PRACTICAL LABORATORY

Build the environment yourself, acquire the data and model, run inference locally, and interpret the evidence.

3.0 Practical map

Part	What you will do	Evidence to retain
3.1	Meet the target and establish a workspace	Board identity and working directory
3.2	Characterise the target resources	Measured hardware/software profile
3.3	Acquire and inspect Fashion-MNIST	Verified files, data properties, sample interpretation
3.4	Install LiteRT and acquire the reference model	Runtime check, model provenance and tensor contract
3.5	Run the first local inference	Prediction, top scores and interpretation
3.6	Evaluate beyond isolated examples	Accuracy, class-level behaviour and errors
3.7	Break the preprocessing contract deliberately	Controlled comparison and explanation
3.8	Close the lab with an evidence record	Reproducibility record and open question for PL2

WORKING PRINCIPLE

Do not optimise for completing commands. At each stage, make a prediction or state an assumption, execute, retain evidence, and explain what the result does and does not support.

WORKING METHOD Treat each shaded code box as one complete copy/paste unit: paste one box at a time, wait for it to finish, then continue to the next box.

Short terminal commands may be typed manually. Longer commands and Python blocks should be copied from this handout to avoid transcription errors. Copying code is not the objective of the laboratory; understanding what each command changes, what each script does and how to interpret the results is.

3.1 Meet the Raspberry Pi and establish your workspace

Treat the Raspberry Pi as the deployment target. Begin by identifying the board and establishing a clean working directory in your user account.

Run in a terminal on the Raspberry Pi

This first check establishes device identity and account provenance. Record the user and hostname now, because later results are only reproducible if you can identify which Raspberry Pi and account produced them.

```
# Identify the current account and Raspberry Pi hostname.
whoami
hostname
```

This block characterises the software context before any package installation. Architecture and Python version directly constrain which pre-built runtime packages can be installed.

```
# Inspect the current location and basic software environment.
pwd
uname -m
python3 --version
```

Create one self-contained workspace for the laboratory rather than scattering files through the home directory. Keeping data, models, scripts and the virtual environment under one project path makes the experiment easier to reproduce and clean up.

```
# Create a dedicated PL1 workspace and enter it.
mkdir -p ~/ai-embedded/pl1
cd ~/ai-embedded/pl1
pwd
ls -lah
```

Terminal reference: `pwd` prints the current directory; `ls -lah` lists files with useful metadata; `cd` changes directory; `mkdir -p` creates a directory path if it does not already exist. The objective is not memorisation: you need enough command-line fluency to inspect and reproduce the deployment environment.

THINK BEFORE YOU CONTINUE

Which elements of this environment are properties of the physical board, which are properties of the operating system, and which can change between two runs of the same model?

RECORD

Group number, board/kit ID, hostname, CPU architecture and Python version. These details belong to the provenance of later measurements.

3.2 Characterise the target resources

A deployment claim such as "the model runs on Raspberry Pi" is incomplete without identifying the target. Characterise the board that you have actually been given; do not copy specifications from the Internet as if they were measurements.

Measure the CPU information reported by the operating system instead of copying a specification sheet. The goal is to distinguish properties of the actual target from assumptions about the product family.

```
# Identify CPU architecture and logical processor count.
uname -m
nproc

# Inspect the CPU description reported by Linux.
lscpu | egrep 'Architecture|Model name|CPU\(s\)' || true
lscpu | grep 'CPU max MHz' || true
```

RAM and persistent storage are different resource budgets. This block records both, together with Python, so that later deployment results have enough context to be interpreted.

```
# Inspect memory, storage and Python version.
free -h
df -h /
python3 --version
```

Temperature is dynamic system state, not a fixed board specification. Capture a baseline value now; later profiling sessions can compare temperature under sustained inference load.

```
# Read CPU temperature when the platform exposes it.
if command -v vcgencmd >/dev/null 2>&1; then
    vcgencmd measure_temp
elif [ -r /sys/class/thermal/thermal_zone0/temp ]; then
    awk '{printf "temp=%.1f C\n", $1/1000}' \
        /sys/class/thermal/thermal_zone0/temp
else
    echo 'CPU temperature: not available'
fi
```

Characteristic	Observed value	Why it may matter for local inference
Architecture	-----	Runtime / wheel compatibility; instruction set
Logical CPUs	-----	Potential parallel execution; not equivalent to linear speed-up
RAM	-----	Model, tensors, runtime and application compete for memory
Free storage	-----	Artifacts, data, logs and environment packages
Python	-----	Runtime package compatibility and reproducibility
CPU temperature	-----	Operating state; potentially relevant under sustained load

A spectrum of embedded and edge targets

MCU

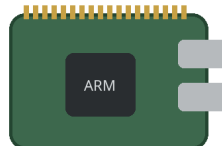
ESP32-S3



- 2 x Xtensa LX7, up to 240 MHz
- 512 KB on-chip SRAM
- Vector extensions for AI/DSP

SBC

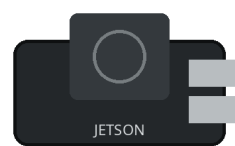
Raspberry Pi 5



- 4 x Arm Cortex-A76, 2.4 GHz
- 1-16 GB LPDDR4X (variant)
- General-purpose Linux SBC

EDGE AI COMPUTER

Jetson Orin Nano Super



- 6 x Arm Cortex-A78AE
- 8 GB LPDDR5
- 67 INT8 TOPS; 7-25 W

Do not rank devices using a single headline number.

Clock frequency, RAM capacity, TOPS, memory bandwidth, accelerators, power envelope and software support describe different constraints. The engineering question is whether a workload satisfies its requirements on a particular target under a controlled protocol.

Reference comparison: ESP32-S3, Raspberry Pi 5 and Jetson Orin Nano Super. Use it to reason about classes of target, not to infer the exact capabilities of your assigned board.

INTERPRET THE COMPARISON

Why is it methodologically weak to conclude that one platform is "better for AI" using only GHz, RAM or TOPS? Identify at least three workload or system properties that would be required for a defensible comparison.

EXPLAIN

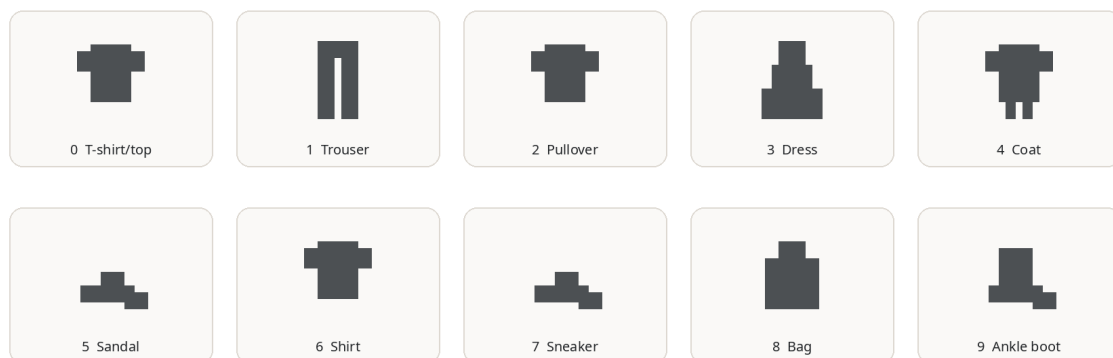
A model file is 20 MB on disk. Explain why this fact alone cannot establish whether inference will fit comfortably in 1 GB of RAM. Your answer should separate persistent storage from runtime memory.

3.3 Fashion-MNIST: acquire the data before touching the model

Why Fashion-MNIST? The aim of PL1 is not to build a useful fashion classifier. We need a compact, controlled vision problem that lets us isolate the deployment pipeline before introducing camera acquisition, annotation, model training and hardware-specific optimisation. Fashion-MNIST gives us known labels, a small image representation and a reproducible test split, so the first questions can focus on data representation, preprocessing, model interfaces and local inference.

A realistic embedded analogue would be a small apparel-sorting or retail intake station: a camera captures a garment, the device crops, resizes and normalises the image, a local classifier estimates the garment category, and the application uses that output to route the item, update inventory or request human verification. Fashion-MNIST is intentionally much simpler than that real system, but the pipeline boundary is the same: sensor data must be transformed into exactly the tensor representation expected by the deployed model.

CAMERA / IMAGE	PREPROCESS	EDGE DEVICE	MODEL OUTPUT	APPLICATION
 Acquire pixels	Crop, resize, scale	Raspberry Pi + LiteRT	Class scores	Route / log / verify



Schematic class silhouettes. The actual dataset samples are inspected on the Raspberry Pi.

Fashion-MNIST contains 28 x 28 grayscale images assigned to ten clothing classes. The examples below are real Fashion-MNIST samples. Their simplicity is useful here: the input is small enough to inspect directly, but still rich enough for genuine classification errors and ambiguity between visually similar classes.



Real Fashion-MNIST samples, enlarged with nearest-neighbour interpolation for visibility. Source dataset: Zalando Research; PNG examples reproduced from a public Fashion-MNIST mirror.

The official dataset contains 60,000 training examples and 10,000 test examples. PL1 downloads only the test images and test labels: about 4.2 MB for the compressed images plus only a few kilobytes for the labels. The training split (about 25 MB compressed) is deliberately not downloaded because you are not training a model yet. This keeps the acquisition step small and makes the purpose of the split explicit.

Download the official test split

Create a dedicated data directory before downloading anything. This keeps provenance clear and makes the following checksum and decoding steps unambiguous.

```
# Return to the PL1 workspace and create a data directory.
cd ~/ai-embedded/pl1
mkdir -p data
```

Download only the official test-image archive because PL1 performs inference, not training. The URL is assembled from short parts so copy/paste from the PDF cannot split a long quoted string.

```
# Download the official Fashion-MNIST test images.
python3 - <<'PY'
from urllib.request import urlretrieve

host = "https://raw.githubusercontent.com"
repo = "/zalandoresearch/fashion-mnist"
folder = "/master/data/fashion"
name = "t10k-images-idx3-ubyte.gz"

url = host + repo + folder + "/" + name
urlretrieve(url, "data/" + name)
print("Downloaded:", name)
PY
```

The label archive is downloaded separately because Fashion-MNIST stores inputs and targets in distinct IDX files. Keeping this separation visible helps distinguish model inputs from ground truth used only for evaluation.

```
# Download the matching test labels.
python3 - <<'PY'
from urllib.request import urlretrieve

host = "https://raw.githubusercontent.com"
repo = "/zalando-research/fashion-mnist"
folder = "/master/data/fashion"
name = "t10k-labels-idx1-ubyte.gz"

url = host + repo + folder + "/" + name
urlretrieve(url, "data/" + name)
print("Downloaded:", name)
PY
```

Before parsing the dataset, verify that the downloaded bytes match the published files. A checksum match is evidence of file identity and transfer integrity; it is not evidence that the dataset represents a real deployment environment.

```
# Check file sizes and verify the official MD5 fingerprints.
ls -lh data/
md5sum data/t10k-images-idx3-ubyte.gz \
      data/t10k-labels-idx1-ubyte.gz
```

VERIFY PROVENANCE

Expected MD5 fingerprints: images `bef4ecab320f06d8554ea6380940ec79`; labels `bb300cfdad3c16e7a12a480ee83cd310`. A matching checksum establishes that the downloaded bytes match the published dataset file. It does not establish that the dataset is representative of a future deployment environment.

Now inspect the binary IDX files rather than relying on a library that hides their structure. Copy the block below as a whole into the terminal.

Inspect shape, dtype, range, class counts and one sample

Read only the binary headers first. This exposes the dataset contract - number of images, spatial dimensions, label count and storage representation - before any model is involved.

```
# Decode only the IDX headers first: structure before content.
python3 - <<'PY'
import gzip
import struct

images = "data/t10k-images-idx3-ubyte.gz"
labels = "data/t10k-labels-idx1-ubyte.gz"

with gzip.open(images, "rb") as f:
    magic, n, rows, cols = struct.unpack(">IIII", f.read(16))

with gzip.open(labels, "rb") as f:
    label_magic, n_labels = struct.unpack(">II", f.read(8))

print("image magic:", magic)
print("image shape:", (n, rows, cols))
print("stored dtype: uint8; nominal range: 0..255")
print("label magic:", label_magic)
print("label shape:", (n_labels,))
print("bytes/image as uint8:", rows * cols)
print("bytes/image as float32:", rows * cols * 4)
PY
```

Now inspect label counts to determine whether the test split is balanced by class. Balance is useful experimental context, but it should not be confused with representativeness of real-world class frequencies.

```
# Inspect the class distribution of the official test split.
python3 - <<'PY'
import gzip
import struct
from collections import Counter

path = "data/t10k-labels-idx1-ubyte.gz"
with gzip.open(path, "rb") as f:
    _, n_labels = struct.unpack(">II", f.read(8))
    labels = list(f.read())

print("labels read:", len(labels), "expected:", n_labels)
print("class counts:", Counter(labels))
PY
```

Render a sample without installing a plotting stack. The ASCII view is intentionally crude: it reminds us that the model receives numeric intensity values, not the semantic object that a human sees.

```
# Render one real image as ASCII directly in the terminal.
python3 - <<'PY'
import gzip
import struct

path = "data/t10k-images-idx3-ubyte.gz"
idx = 25
chars = " .:-+*#%@"

with gzip.open(path, "rb") as f:
    _, n, rows, cols = struct.unpack(">IIII", f.read(16))
    pixels = f.read()

size = rows * cols
sample = pixels[idx * size:(idx + 1) * size]
print("sample:", idx, "of", n)

for r in range(0, rows, 2):
    row = sample[r * cols:(r + 1) * cols]
    print("".join(chars[min(9, p * 10 // 256)] for p in row))
PY
```

Observation	Your evidence
Image tensor shape	_____
Original dtype and numeric range	_____
Label tensor shape	_____
Class distribution in the test split	_____
Bytes per image as uint8 / float32	_____
What the ASCII representation reveals / hides	_____

INTERPRET THE DATA

Fashion-MNIST is balanced by class, but balance is not the same as representativeness. Identify at least three deployment variables that this benchmark removes or fixes - for example viewpoint, illumination, background, sensor characteristics, garment deformation or class prevalence - and explain how each could alter the error distribution of a real embedded system.

TRANSFER TO A REAL SYSTEM

Imagine replacing Fashion-MNIST with a 1920 x 1080 RGB camera. Which preprocessing decisions would become part of the deployment contract before a model could receive a tensor? Be precise about geometry, channels, numeric representation and normalisation.

3.4 LiteRT and the reference model: acquire the runtime and artifact

The reference model was trained elsewhere. That separation is intentional: PL1 starts at deployment, while later practical work gives you control of training and conversion. Every group now creates the same isolated environment, installs the same LiteRT release, downloads the same pinned model artifact and inspects the interface exposed by that artifact. Do not skip the installation block because a package appears to be present globally: reproducibility requires the laboratory environment to be explicit. The dataset and model downloads used in PL1 are deliberately small; the first LiteRT environment installation is the network-heavy part of the setup.

Term	Meaning	PL1 example
Tensor	A multidimensional array of numeric values passed between operations in a model.	One image becomes the input tensor; ten scores form the output tensor.
Shape	The size of each tensor dimension. The leading dimension is often the batch size.	`[1, 28, 28]` means one 28 x 28 grayscale image.
dtype	The numeric representation used by each tensor element.	`float32` uses 32-bit floating-point values; `uint8` uses 8-bit unsigned integers.
Quantisation parameters	Scale and zero-point metadata used when integer values represent a real-valued range.	A float model may report no meaningful quantisation; later PLs study INT8 explicitly.
Input / output tensor	The runtime-visible boundary through which application data enters and model results leave.	The application must match the expected shape and dtype exactly.

Create one common environment - every group runs this block

Prepare the Python tooling required for an isolated environment. Re-running this package-manager command is safe: packages that are already installed are left in place.

```
# Work inside the PL1 directory.
cd ~/ai-embedded/pl1

# Ensure virtual-environment support and pip are available.
sudo apt-get install -y python3-venv python3-pip
```

Create a virtual environment so every Python dependency for the laboratory is isolated from the operating-system Python. After activation, verify that pip points inside `.venv` before installing LiteRT.

```
# Create and activate a project-local Python environment.
python3 -m venv .venv
source .venv/bin/activate

# Confirm that pip now belongs to the virtual environment.
python -m pip --version
```

Install one pinned LiteRT release for every group. Pinning the runtime removes a major source of experimental drift; NumPy is constrained to a compatible major-version range.

```
# Install the same LiteRT release in every group environment.
python -m pip install --disable-pip-version-check \
    'ai-edge-litert==2.2.0' \
    'numpy<3'
```

A successful import is the cheapest runtime sanity check. It proves that the Python environment can load the installed packages, but it does not yet prove that the model artifact is compatible or semantically correct.

```
# Verify that the required packages can be imported.
python - <<'PY'
import numpy as np
from ai_edge_litert.interpreter import Interpreter

print("NumPy:", np.__version__)
print("LiteRT Interpreter import: OK")
PY
```

IF ENVIRONMENT CREATION FAILS

Do not repair a packaging problem with `sudo pip`. Confirm `uname -m` and `python3 --version`, then read the error message. The laboratory assumes a 64-bit Raspberry Pi OS and a Python version supported by `ai-edge-litert`. Report incompatibilities rather than modifying the global Python environment.

Download the reference model from a pinned public GitHub commit. The `.tflite` file is about 1.6 MB, so model acquisition is intentionally small. Pinning the commit makes the artifact reproducible: if the upstream project changes later, this laboratory still refers to the same bytes and training provenance.

Acquire and fingerprint the model artifact

Create a model directory so the deployment artifact has an explicit location separate from data and scripts. This also makes model provenance and hashing easier to record.

```
# Create a local directory for the deployment artifact.
mkdir -p model
```

Download the exact model revision used by the whole class. The Git commit is pinned, so every group receives the same bytes even if the source repository changes later.

```
# Download the pinned reference model.
# Short URL parts avoid line-wrap errors when copying from the PDF.
python - <<'PY'
from urllib.request import urlretrieve

host = "https://raw.githubusercontent.com"
repo = "/LycusCoder/Mini-Vision_System-_Fashion-MNIST_Classifier"
commit = "/8e480da0b55b286fbb7dd48201c7bcb660e6bb70"
file = "/models/fashion_mnist_mlp.tflite"
target = "model/fashion_mnist_mlp.tflite"

url = host + repo + commit + file
urlretrieve(url, target)
print("Model downloaded to:", target)
PY
```

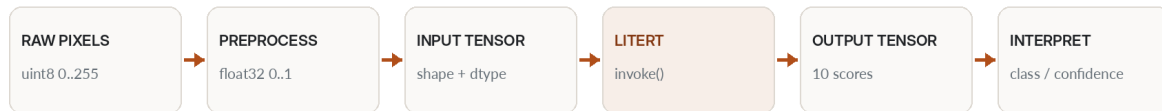
Record both model size and SHA-256 fingerprint. Size is a deployment property; the hash is a provenance property that lets two groups prove they evaluated the same artifact.

```
# Record artifact size and a cryptographic fingerprint.
ls -lh model/fashion_mnist_mlp.tflite
sha256sum model/fashion_mnist_mlp.tflite
```

The source project trains a simple Fashion-MNIST MLP with input shape 28 x 28, a 512-unit ReLU hidden layer and a 10-class softmax output. Training code normalises pixels to the 0..1 range before export. Treat that provenance as part of the deployment contract, not as incidental documentation.

Inference is a contract, not a function call

A model can execute without error and still be semantically wrong if any stage disagrees with training-time assumptions.



PL1 experiment: keep the model fixed and intentionally violate preprocessing. If predictions change, the failure belongs to the pipeline, not to the CPU.

Inspect the model rather than guessing its interface

Ask LiteRT to parse the `.tflite` artifact and allocate the tensors/buffers required by its graph. Success establishes a useful compatibility milestone, but it is still not an end-to-end inference validation.

```
# Load the model and allocate its runtime tensors.
python - <<'PY'
from ai_edge_litert.interpreter import Interpreter

model = "model/fashion_mnist_mlp.tflite"
interpreter = Interpreter(model_path=model)
interpreter.allocate_tensors()
print("Model loaded and tensors allocated")
PY
```

Inspect the runtime-visible model contract before supplying data. Shape tells you the required tensor dimensions, dtype tells you the numeric representation, and quantization metadata indicates whether affine integer quantization parameters are present.

```
# Inspect the externally visible input/output tensor contract.
python - <<'PY'
from ai_edge_litert.interpreter import Interpreter

model = "model/fashion_mnist_mlp.tflite"
interpreter = Interpreter(model_path=model)
interpreter.allocate_tensors()
input_info = interpreter.get_input_details()[0]
output_info = interpreter.get_output_details()[0]

print("\nINPUT")
print(" name:", input_info["name"])
print(" shape:", input_info["shape"])
print(" dtype:", input_info["dtype"])
print(" quantization:", input_info.get("quantization"))

print("\nOUTPUT")
print(" name:", output_info["name"])
print(" shape:", output_info["shape"])
print(" dtype:", output_info["dtype"])
print(" quantization:", output_info.get("quantization"))
PY
```

INTERPRET THE ARTIFACT

Separate information encoded in the runtime-visible tensor interface from assumptions that remain external to the file. Which of shape, dtype, class semantics, pixel scaling, channel order, label mapping, training distribution and decision thresholds can be inferred safely from the artifact alone? Explain why provenance is part of a deployable system.

EXPLAIN

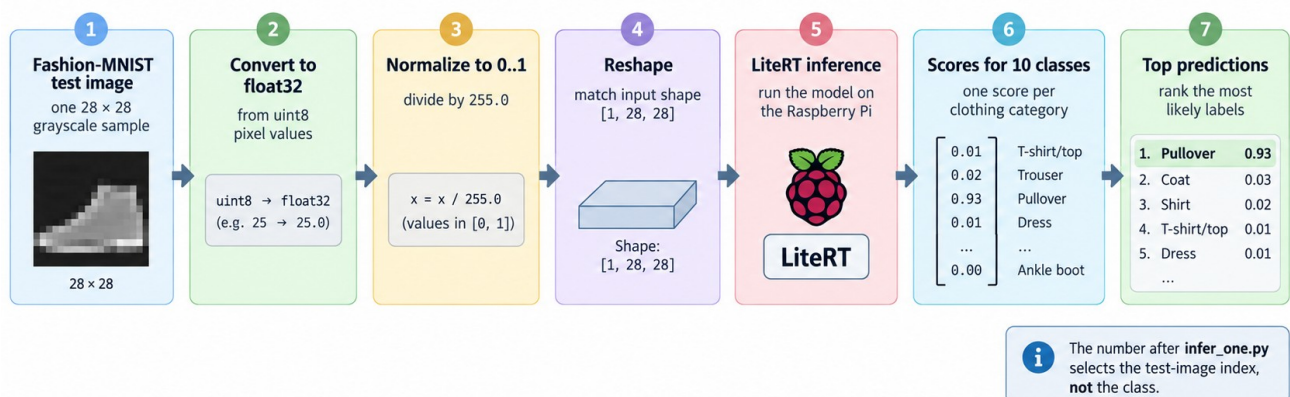
The interpreter loads and allocates tensors successfully. Separate what this proves about software compatibility from what it does not prove about predictive correctness, preprocessing, robustness or suitability for a real deployment.

3.5 Run your first local inference

The first inference closes the path from compressed dataset bytes to a local prediction. The code deliberately exposes each boundary: decode the stored image, convert its numeric representation, reproduce the training-time scaling, reshape it to the model input tensor, copy it into LiteRT, invoke the graph and interpret the output scores. In production these steps may be hidden inside an application pipeline; here they remain visible so that integration errors can be reasoned about.

Inference Pipeline for One Fashion-MNIST Sample

From image to predicted clothing label (on an embedded device)



Inference pipeline used throughout Section 3.5: the same sample passes through numeric conversion, preprocessing, tensor-shape adaptation and LiteRT before the output scores are interpreted.

Run an inference for sample 25

SAMPLE INDEX. The number after `infer_one.py` is the index of the test image to classify. It is not a class ID. For example, `python infer_one.py 25` selects test-set sample 25 (the 26th sample because indexing starts at zero).

Build the reusable part of the inference program first: imports, class names and IDX decoding helpers. Keeping these definitions in a module allows later scripts to reuse the exact same data-loading logic.

```
# Create a reusable inference script: imports, labels and IDX loaders.
cat > infer_one.py <<'PY'
import gzip
import struct
import sys
import numpy as np
from ai_edge_litert.interpreter import Interpreter

LABELS = [
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot",
]

def load_images(path):
    f = gzip.open(path, "rb")
    _, n, rows, cols = struct.unpack(">IIII", f.read(16))
    data = np.frombuffer(f.read(), dtype=np.uint8)
    f.close()
    return data.reshape(n, rows, cols)

def load_labels(path):
    f = gzip.open(path, "rb")
    f.read(8)
    data = np.frombuffer(f.read(), dtype=np.uint8)
    f.close()
    return data
PY
```

Add the end-to-end inference function in a second, smaller block. It performs the complete path for one sample: load data, apply the training-time scale, reshape to the input tensor, invoke LiteRT and rank the top three outputs.

```
# Append one inference function. Every line inside the function has one indentation level.
cat >> infer_one.py <<'PY'

def predict(index):
    X = load_images("data/t10k-images-idx3-ubyte.gz")
    y = load_labels("data/t10k-labels-idx1-ubyte.gz")
    runtime = Interpreter(model_path="model/fashion_mnist_mlp.tflite")
    runtime.allocate_tensors()
    input_info = runtime.get_input_details()[0]
    output_info = runtime.get_output_details()[0]
    x = X[index].astype(np.float32) / 255.0
    x = x.reshape(tuple(int(v) for v in input_info["shape"]))
    runtime.set_tensor(input_info["index"], x)
    runtime.invoke()
    scores = runtime.get_tensor(output_info["index"])[0]
    order = np.argsort(scores)[::-1][:3]
    truth = int(y[index])
    print("true:", truth, LABELS[truth])
    print(1, int(order[0]), LABELS[int(order[0])], float(scores[order[0]]))
    print(2, int(order[1]), LABELS[int(order[1])], float(scores[order[1]]))
    print(3, int(order[2]), LABELS[int(order[2])], float(scores[order[2]]))
PY
```

Finish the script with an import-safe command-line entry point. The `\_\_main\_\_` guard is essential because later evaluation scripts import the loader functions; imported modules must not try to read a missing command-line argument.

```
# Add an import-safe command-line entry point.
cat >> infer_one.py <<'PY'

if __name__ == "__main__":
    if len(sys.argv) != 2:
        raise SystemExit("Usage: python infer_one.py <sample_index>")
    predict(int(sys.argv[1]))
PY

# Run the same reproducible example used in this handout.
python infer_one.py 25
```

Repeat the exact same pipeline on several fixed indices. Comparing a few examples is useful for debugging and interpretation, but it is not a statistically defensible model evaluation.

```
# Repeat deliberately with different examples.
python infer_one.py 0
python infer_one.py 1
python infer_one.py 42
```

REPEAT DELIBERATELY

Change only `idx` and test several samples. Retain at least one correct and, if you encounter one, one incorrect prediction. Do not cherry-pick examples to make the model appear stronger or weaker.

INTERPRET THE SCORES

A high softmax score is not automatically a calibrated probability of correctness. What additional experiment would be required before you could make a claim such as "a score of 0.9 means the model is correct approximately 90% of the time"?

3.6 Evaluate beyond isolated examples

A few examples are useful for debugging the pipeline; they are weak evidence for model quality. Evaluate a fixed subset and inspect the structure of its errors.

Evaluate a fixed subset and retain the output

Create a separate evaluator that reuses the tested data loaders but owns one LiteRT runtime instance. Reusing the runtime avoids paying model-loading overhead for every sample and keeps the comparison protocol consistent.

```
# Create an evaluation script and reuse the tested IDX helpers.
cat > evaluate_subset.py <<'PY'
import numpy as np
from ai_edge_litert.interpreter import Interpreter
from infer_one import LABELS, load_images, load_labels

X = load_images("data/t10k-images-idx3-ubyte.gz")
y = load_labels("data/t10k-labels-idx1-ubyte.gz")
runtime = Interpreter(model_path="model/fashion_mnist_mlp.tflite")
runtime.allocate_tensors()
input_info = runtime.get_input_details()[0]
output_info = runtime.get_output_details()[0]
shape = tuple(int(v) for v in input_info["shape"])

def infer_scores(x):
    runtime.set_tensor(input_info["index"], x.reshape(shape))
    runtime.invoke()
    return runtime.get_tensor(output_info["index"])[0].copy()
PY
```

Evaluate the same first 500 test examples for every group. The script records overall accuracy, a confusion matrix and the most confident mistakes so that model quality is discussed structurally rather than as one scalar. The printed matrix uses true classes as rows and predicted classes as columns. The script also prints the class-index order immediately below the matrix. A diagonal cell therefore counts correct predictions for that class, while an off-diagonal cell counts one specific type of confusion (for example, true Shirt predicted as T-shirt/top).

```
# Evaluate one fixed subset and report accuracy, confusion and confident errors.
cat >> evaluate_subset.py <<'PY'

limit = 500
score_rows = np.stack([infer_scores(X[i].astype(np.float32) / 255.0) for i in range(limit)])
truth = y[:limit].astype(int)
predicted = np.argmax(score_rows, axis=1)
confusion = np.zeros((10, 10), dtype=int)
np.add.at(confusion, (truth, predicted), 1)
accuracy = float(np.mean(predicted == truth))
error_idx = np.where(predicted != truth)[0]
error_scores = score_rows[error_idx, predicted[error_idx]]
ranked = error_idx[np.argsort(error_scores)[:5]]

print("accuracy:", accuracy)
print("confusion matrix (rows=true, cols=pred):")
print(confusion)
print("class index order:")
print("0 T-shirt/top | 1 Trouser | 2 Pullover | 3 Dress | 4 Coat")
print("5 Sandal | 6 Shirt | 7 Sneaker | 8 Bag | 9 Ankle boot")
print("Rows = true class; columns = predicted class")
print("highest-confidence errors:")
records = [
    (float(score_rows[i, predicted[i]]), int(i), int(truth[i]), int(predicted[i]))
    for i in ranked
]
print(records)
PY
```

Run the evaluator only after the script has been fully assembled. Keep the printed output: it becomes the evidence used in the analysis questions immediately below.

```
# Execute the evaluation script.
python evaluate_subset.py
```

ANALYSE

Which class pairs dominate the off-diagonal entries? Propose a data-based explanation and a model-based explanation. What evidence would distinguish between the two?

METHODOLOGICAL QUESTION

If another group reports a different accuracy after evaluating a different subset, are the two numbers directly comparable? State the conditions under which a comparison would become defensible.

3.7 Break the preprocessing contract deliberately

Now run a controlled failure experiment. Hold the model, examples and runtime constant. Change only preprocessing. This is a small example of a central embedded-ML lesson: a correct model artifact inside an incorrect pipeline can produce convincing but invalid outputs.

BEFORE EXECUTING

Predict what will happen under three input scales: (A) the training-time 0..1 scale; (B) raw 0..255 values cast to Float32; and (C) dividing by 255 twice. Do not assume that every contract violation must reduce top-1 accuracy - write a hypothesis about both accuracy and score behaviour before you run the experiment.

Controlled preprocessing experiment

Create a controlled experiment in which model, data subset and runtime remain fixed. Only the numeric scale presented to the model will change, making preprocessing the independent variable.

```
# Create the controlled preprocessing experiment.
cat > preprocessing_test.py <<'PY'
import numpy as np
from ai_edge_litert.interpreter import Interpreter
from infer_one import load_images, load_labels

X = load_images("data/t10k-images-idx3-ubyte.gz")
y = load_labels("data/t10k-labels-idx1-ubyte.gz")
runtime = Interpreter(model_path="model/fashion_mnist_mlp.tflite")
runtime.allocate_tensors()
input_info = runtime.get_input_details()[0]
output_info = runtime.get_output_details()[0]
shape = tuple(int(v) for v in input_info["shape"])

def infer_class(x):
    runtime.set_tensor(input_info["index"], x.reshape(shape))
    runtime.invoke()
    return int(np.argmax(runtime.get_tensor(output_info["index"])[0]))
PY
```

Add a compact evaluator that receives only a scale factor. Because all other code paths are shared, differences in the result can be attributed more cleanly to the changed input representation.

```
# Append a copy-safe evaluator: the scale is the only experimental variable.
cat >> preprocessing_test.py <<'PY'

def evaluate(scale, limit=300):
    predicted = [infer_class(X[i].astype(np.float32) * scale) for i in range(limit)]
    return float(np.mean(np.asarray(predicted) == y[:limit]))
PY
```

Run all three conditions on the same 300 examples. The point is not to force a particular ranking; it is to observe whether a contract violation is obvious, subtle or silent under this particular model.

```
# Run the three preprocessing conditions on the same subset.
cat >> preprocessing_test.py <<'PY'

print("correct", evaluate(1.0 / 255.0))
print("raw", evaluate(1.0))
print("double", evaluate(1.0 / (255.0 * 255.0)))
PY

python preprocessing_test.py
```

Condition	Prediction before running	Observed accuracy	Interpretation
Correct 0..1	-----	-----	----- --
Raw 0..255	-----	-----	----- --

Divide twice	-----	-----	-----
			--

Analysis guidance. Compare the three preprocessing conditions using both the measured accuracy and the model scores. Do not assume beforehand that every contract violation must reduce top-1 accuracy. If an apparently incorrect pipeline performs similarly on this subset, treat that as a result to explain rather than as proof that the pipeline is valid. Ask what other evidence would be needed to establish semantic equivalence and safe deployment.

DEPLOYMENT IMPLICATION

In a production system, how would you make the preprocessing contract versioned, testable and difficult to violate silently? Propose at least two engineering mechanisms.

3.8 Close the laboratory with an evidence record

A reproducible deployment is more than a screenshot of a successful prediction. Complete the record below so that another group could understand what you ran and identify what remains uncertain.

Evidence item	Your record
Board identity / architecture / Python	-----
Dataset source + checksum status	-----
Model source + SHA-256	-----
Model input shape / dtype	-----
Model output shape / dtype	-----
Preprocessing used for valid inference	-----
Evaluation subset and main result	-----
Most informative error or confusion	-----
Observed effect of preprocessing mismatch	-----

FINAL SYNTHESIS

Explain the complete path from a Fashion-MNIST file on disk to a predicted class. Your explanation must explicitly name the data representation, preprocessing, input tensor, LiteRT runtime, output tensor and interpretation stage.

BRIDGE TO PL2

What is the most important question that PL1 cannot answer because the model was provided already trained? Phrase it as a question that could be investigated when you gain control of model architecture and training.

4

OPTIONAL AI EXPLORATIONS

For groups that finish the core path early: run complete, standalone AI experiments and extend the evidence about model behaviour, not the device setup.

For groups that finish the core path early: extend the evidence about model behaviour, not the device setup.

Before the AI explorations: connect to your Raspberry Pi by SSH

Connect your own computer to the Wi-Fi router/network provided for the Raspberry Pi laboratory. Ask the instructor for the network name (SSID) and access details. Your computer and the Raspberry Pi must be on the same local network before SSH can connect.

Before using SSH, discover the address assigned to the Raspberry Pi on the laboratory network. Record the IPv4 address that belongs to the router/network indicated by the instructor.

```
# On the Raspberry Pi, show its assigned IP addresses.
hostname -I
```

Use this only if `hostname -I` shows several addresses or if connectivity is unclear. `ip addr` exposes the interface names and addresses so you can identify the active network path.

```
# Optional: inspect the network interfaces in more detail.
ip addr
```

RECORD

Raspberry Pi IP address: _____

Now move to a terminal on your own computer. Replace <RASPBERRY_PI_IP> with the address that you recorded above, then connect using the laboratory account:

Run this command on your own laptop, not on the Raspberry Pi. SSH opens an encrypted remote shell to the Raspberry Pi using the shared laboratory account.

```
# Replace YOUR_PI_IP with the address recorded above.
ssh iase@YOUR_PI_IP
```

FIRST CONNECTION

SSH may ask whether you trust the host key. Confirm that you are connecting to your assigned Raspberry Pi, then type yes. When prompted for the password, enter iase2026. Password characters are not echoed in the terminal; this is normal.

Once connected, verify that the remote shell belongs to the expected machine and user. Use exit when you want to return to the terminal on your own computer.

Once connected, verify that the remote hostname and user match the assigned board before running any commands. `exit` closes the SSH session and returns you to your laptop shell.

```
# Verify the remote machine and account.
hostname
whoami

# End the SSH session and return to your own computer.
exit
```

These explorations are intentionally centred on AI behaviour. Choose one or more only after the core evidence record is complete. Each extension should start with a hypothesis, change one controlled factor, retain the result and state the limits of the conclusion. The code in this section is provided as complete standalone scripts, split into small copy-safe blocks. Paste each block in order. Keeping the dataset, model, runtime and comparison protocol unchanged - except for the stated perturbation - is part of the experimental control.

4.1 Occlusion sensitivity

A controlled occlusion removes information from a fixed region while preserving the model, runtime and original sample. The effect is not necessarily monotonic: less visible information does not guarantee lower confidence, and the important task is to compare the baseline and perturbed outputs before proposing an explanation.

This extra turns digital occlusion into a complete, standalone experiment rather than a Python fragment. The first block loads exactly the same data/model contract as the core path and defines one reusable prediction function.

```
# Build a standalone occlusion experiment: setup and reusable prediction function.
cat > occlusion_test.py <<'PY'
import numpy as np
from ai_edge_litert.interpreter import Interpreter
from infer_one import load_images, load_labels, LABELS

X = load_images("data/t10k-images-idx3-ubyte.gz")
y = load_labels("data/t10k-labels-idx1-ubyte.gz")
idx = 25

runtime = Interpreter(model_path="model/fashion_mnist_mlp.tflite")
runtime.allocate_tensors()
input_info = runtime.get_input_details()[0]
output_info = runtime.get_output_details()[0]
shape = tuple(int(v) for v in input_info["shape"])

def predict(image):
    x = image.astype(np.float32) / 255.0
    runtime.set_tensor(input_info["index"], x.reshape(shape))
    runtime.invoke()
    return runtime.get_tensor(output_info["index"])[0].copy()
PY
```

The second block changes only one factor: a fixed central 12×12 region is replaced with zero-valued pixels. Compare both the class and the maximum score; do not assume that occlusion must reduce confidence.

```
# Append the controlled occlusion and compare original vs masked input.
cat >> occlusion_test.py <<'PY'
baseline_scores = predict(X[idx])

occluded = X[idx].copy()
occluded[8:20, 8:20] = 0
occluded_scores = predict(occluded)

baseline_class = int(np.argmax(baseline_scores))
occluded_class = int(np.argmax(occluded_scores))

print("True class:", int(y[idx]), LABELS[int(y[idx])])
print("Original class:", baseline_class, LABELS[baseline_class])
print("Original score:", float(baseline_scores[baseline_class]))
print("Occluded class:", occluded_class, LABELS[occluded_class])
print("Occluded score:", float(occluded_scores[occluded_class]))
PY

python occlusion_test.py
```

RESEARCH QUESTION

Does the effect of a fixed occlusion depend systematically on the true class? Design a comparison that would support or reject that claim.

4.2 Noise robustness

Add controlled random noise to the normalised input and measure how accuracy changes. Use a fixed random seed so that the experiment can be repeated. Compare multiple noise amplitudes and clip the result to the valid 0..1 range.

This experiment asks how the classifier behaves when the input is corrupted by reproducible Gaussian noise. The first block fixes the sample, model, runtime and random seed so that noise magnitude is the only changing factor.

```
# Build a standalone noise experiment: setup and reusable prediction function.
cat > noise_test.py <<'PY'
import numpy as np
from ai_edge_litert.interpreter import Interpreter
from infer_one import load_images, load_labels, LABELS

X = load_images("data/t10k-images-idx3-ubyte.gz")
y = load_labels("data/t10k-labels-idx1-ubyte.gz")
idx = 25

runtime = Interpreter(model_path="model/fashion_mnist_mlp.tflite")
runtime.allocate_tensors()
input_info = runtime.get_input_details()[0]
output_info = runtime.get_output_details()[0]
shape = tuple(int(v) for v in input_info["shape"])
rng = np.random.default_rng(42)

def predict_normalized(image):
    x = image.astype(np.float32).reshape(shape)
    runtime.set_tensor(input_info["index"], x)
    runtime.invoke()
    return runtime.get_tensor(output_info["index"])[0].copy()
PY
```

The second block keeps the exact same image and increases only the noise standard deviation. Track when the class changes and how the top score evolves; robustness is about behaviour under controlled perturbation, not just one clean accuracy value.

```
# Append the baseline and progressively noisier conditions.
cat >> noise_test.py <<'PY'
baseline = X[idx].astype(np.float32) / 255.0
baseline_scores = predict_normalized(baseline)
baseline_class = int(np.argmax(baseline_scores))

print("True class:", int(y[idx]), LABELS[int(y[idx])])
print("Baseline class:", baseline_class, LABELS[baseline_class])
print("Baseline score:", float(baseline_scores[baseline_class]))

for sigma in [0.05, 0.10, 0.20, 0.30]:
    noise = rng.normal(0.0, sigma, baseline.shape)
    noisy = np.clip(baseline + noise, 0.0, 1.0)
    scores = predict_normalized(noisy)
    predicted = int(np.argmax(scores))
    print("sigma =", sigma)
    print(" class =", predicted, LABELS[predicted])
    print(" score =", float(scores[predicted]))
PY

python noise_test.py
```

INTERPRETATION CHALLENGE

A monotonic drop in accuracy would be plausible, but do not assume it. If a small amount of noise improves a few predictions, how would you report that without claiming that noise is beneficial?

4.3 Confidence and high-confidence errors

Collect the maximum output score for correct and incorrect predictions. Compare their distributions or summary statistics. Identify examples where the model is wrong with a high score.

RESEARCH QUESTION

Can maximum softmax score be used as a reliable rejection threshold for this model? What evidence would you need to choose and validate such a threshold?

4.4 Per-class behaviour and confusion structure

Use the confusion matrix from Section 3.6 to calculate per-class recall and inspect the most frequent confusions. Move beyond global accuracy: a model can have acceptable average performance while systematically failing a particular class.

ENGINEERING QUESTION

Suppose confusing Shirt with T-shirt/top has low application cost, while confusing Ankle boot with Sandal has high cost. How should that change the evaluation metric and final decision?

4.5 Design your own controlled perturbation

Propose one perturbation that remains relevant to the input or model behaviour: contrast change, partial cropping, translation, label mapping error, altered thresholding or another justified condition. Before writing code, complete the design.

Experimental element	Your design
Question	-----
Hypothesis	-----
Independent variable	-----
What remains constant	-----
Evidence / metric	-----
Result that would change your interpretation	-----
Main limitation	-----

R

TECHNICAL SOURCES AND PROVENANCE

Selected references supporting the terminology, dataset, platforms and reference model used in this handout.

[1] LiteRT / Keras export guide

Current Python inference examples use ``ai_edge_litert.interpreter.Interpreter`` and ``tflite`` artifacts. [Open source](https://keras.io/guides/litert_export/)
https://keras.io/guides/litert_export/

[2] Fashion-MNIST official repository

Dataset definition, label mapping, test files and published checksums. [Open source](https://github.com/zalando-research/fashion-mnist)
<https://github.com/zalando-research/fashion-mnist>

[3] Raspberry Pi 5 product specification

Reference SBC specifications used in the hardware comparison. [Open source](https://www.raspberrypi.com/products/raspberry-pi-5/)
<https://www.raspberrypi.com/products/raspberry-pi-5/>

[4] NVIDIA Jetson Orin Nano Super

Reference edge AI computer specifications. [Open source](https://www.nvidia.com/en-eu/autonomous-machines/embedded-systems/jetson-orin/nano-super-developer-kit/)
<https://www.nvidia.com/en-eu/autonomous-machines/embedded-systems/jetson-orin/nano-super-developer-kit/>

[5] Espressif ESP32-S3

Reference MCU specifications and vector acceleration features. [Open source](https://www.espressif.com/en/products/socs/esp32-s3)
<https://www.espressif.com/en/products/socs/esp32-s3>

[6] Reference Fashion-MNIST model repository

Pinned source of the model artifact and training script used in PL1. [Open source](https://github.com/LycusCoder/Mini-Vision_System-_Fashion-MNIST_Classifier/tree/8e480da0b55b286fbb7dd48201c7bcb660e6bb70)
https://github.com/LycusCoder/Mini-Vision_System-_Fashion-MNIST_Classifier/tree/8e480da0b55b286fbb7dd48201c7bcb660e6bb70

[7] ISEP brand resources

Institutional brand resources and official wordmarks. [Open source](https://www.isep.ipp.pt/isep/docpub)
<https://www.isep.ipp.pt/isep/docpub>

[PyTorch -> LiteRT] Google AI Edge, PyTorch-to-LiteRT conversion tooling (LiteRT Torch / AI Edge Torch). <https://ai.google.dev/edge/litert>

SOURCE DISCIPLINE

PL1 uses a pinned reference model because model provenance is part of deployment reproducibility. The dataset is obtained from the official Fashion-MNIST repository and verified using the published checksums.